Ejercicios de clases y objetos

El ejercicio 5 continúa el 4; el ejercicio 6 retoma las clases del 3.

Define primero cada clase completa, con sus métodos dentro. Después, fuera de la clase, crea los objetos y llama a sus métodos para probarlos.

1. Crear una clase y sus objetos

Define la clase **Coche**. Dentro de ella, escribe un __**init**__ que reciba la marca y el modelo y los guarde como atributos de cada objeto.

Una vez definida la clase, fuera de ella:

- Crea dos objetos que representen coches distintos.
- Muestra la marca y el modelo de cada uno.
- Cambia el modelo del primer coche y comprueba que el segundo conserva el suyo.

2. Modificar el estado de un objeto

Escribe la definición completa de la clase **CochePruebas**. Los tres métodos siguientes deben quedar definidos **dentro del cuerpo de la clase**:

- __**init**__**()**: crea el atributo de instancia **velocidad** con valor inicial 0, en km/h.
- **acelerar()**: aumenta en 10 la velocidad del propio objeto.
- **detener()**: pone a cero la velocidad del propio objeto.

Después de terminar la definición de la clase, **fuera de ella**, crea dos coches de pruebas. Llama a sus métodos para acelerarlos un número distinto de veces y detén solo uno. Comprueba que cada coche conserva su propia velocidad y que detener uno no cambia la del otro.

3. Heredar atributos y métodos

Define una clase **Vehiculo**. Su inicializador debe guardar **marca** y **modelo**, y su método **describir()** debe devolver un texto que incluya ambos atributos.

Define después una clase **CocheElectrico** que herede de **Vehiculo**. Dentro de ella, escribe un inicializador que reutilice el de **Vehiculo** mediante **super()** y guarde también **autonomia_km**. Utiliza **describir()** por herencia, sin volver a definirlo.

Fuera de las clases, crea un coche eléctrico y comprueba que puedes consultar sus tres atributos y utilizar la descripción heredada.

4. Sobrescribir un método heredado

Define una clase **Motor** para representar motores de coches. Su inicializador debe guardar **modelo** y su método **arrancar()** debe devolver **"Motor en marcha"**. Crea dos subclases:

- **MotorGasolina**: su método **arrancar()** debe devolver **"Motor de gasolina en marcha"**.
- **MotorElectrico**: su método **arrancar()** debe devolver **"Motor eléctrico en marcha"**.

Ambas subclases deben utilizar el inicializador heredado de **Motor**, sin definir otro __**init**__. Después, crea un motor de cada tipo con modelos distintos y comprueba sus modelos y mensajes de arranque.

5. Utilizar el polimorfismo

Reutiliza las clases del ejercicio anterior. Escribe una función **poner_en_marcha(motor)** que llame al método **arrancar()** del objeto recibido y devuelva su resultado. Después:

- Crea una lista con un objeto **MotorGasolina** y otro **MotorElectrico**.
- Recorre la lista con un bucle y utiliza **poner_en_marcha()** para mostrar el mensaje de cada motor.

La función debe funcionar sin comprobar el tipo del motor con **type()** o **isinstance()**.

Finalmente, añade una clase **MotorDiesel** que herede de **Motor** y cuyo método **arrancar()** devuelva **"Motor diésel en marcha"**. Incorpora un motor diésel a la lista y comprueba que funciona **sin modificar la función poner_en_marcha() ni el bucle**.

6. Ampliar un método heredado con super()

Retoma las clases del ejercicio 3. Escribe una nueva versión de **CocheElectrico** que conserve su inicializador y defina su propio método **describir()**.

Ese método debe aprovechar la descripción de **Vehiculo** mediante **super()**, añadir una frase con la autonomía en kilómetros y devolver el mensaje completo. Reutiliza la descripción original sin volver a escribirla manualmente.

Una vez definida esta versión de la clase, crea un coche eléctrico y comprueba tanto la descripción ampliada como la conservación de sus atributos.

7. Mostrar datos con f-strings

Utiliza estos datos:

```
marca = "Seat"
modelo = "Ibiza"
anio = 2020
```

Escribe un único **print()** con una **f-string** que inserte los valores de las tres variables en una frase legible.

Después, cambia los valores de las variables y comprueba que el mensaje se actualiza sin modificar el **print()**. No necesitas crear ninguna clase.

8. Gestionar una entrada incorrecta con try y except

Pide al usuario los kilómetros recorridos por un coche con **input()** e intenta convertirlos a un entero con **int()** dentro de un bloque **try**.

- Si la conversión funciona, muestra **"Kilómetros indicados:"** seguido del número.
- Si se produce **ValueError**, muestra **"Debes escribir un número entero."**.
- Fuera del **try / except**, muestra siempre **"Programa finalizado."**.

Haz dos pruebas en ejecuciones distintas: introduce primero **12000** y después **doce mil**. Comprueba que el programa muestra el mensaje adecuado y llega al mensaje final en ambos casos.

Solo debes comprobar si la entrada puede convertirse a entero. No hace falta repetir la pregunta ni utilizar un bucle.